

CPU, Processor, and Cores

CPU (Central Processing Unit)

- It's the **brain of the computer**.
- It **processes all instructions** from software and hardware.
- Also called the **Processor**.

Processor

- Another name for **CPU**.
- Handles all **calculations and operations** in the system.
- Determines the **speed and performance** of a computer.

Cores

- A **core** is a **single processing unit** inside the CPU.
- **More cores = More tasks at the same time** (better multitasking).
- Example:
 - **Dual-core** = 2 cores
 - **Quad-core** = 4 cores
 - **Octa-core** = 8 cores

Trick to Remember

- **CPU = Brain**
 - **Processor = Worker**
 - **Cores = Number of brains/workers inside**
-

OS and Kernel

Operating System (OS)

- The **main software** that controls the computer.
- Manages **hardware, software, files, and resources**.
- Examples: **Windows, Linux, macOS, Android**.

Kernel

- The **core part of the OS**.
- Connects **hardware** and **software**.
- Handles tasks like **CPU, memory, and device management**.

Trick to Remember

- **OS = Full system manager**

- **Kernel = Core brain of the OS**
-

Process

What is a Process?

- A **program in execution**.
- It has its own **memory, code, data, and system resources**.
- Example: Opening Chrome = starting a new process.

Trick to Remember

- **Process = Running program**
 - Each time you open a program, a **new process** is created.
-

Thread, Concurrency, and Parallelism

Thread

- A **small unit of a process**.
- Think of it as a **single line of work** in a program.
- A process can have **multiple threads** working together.

Concurrency

- **Handling multiple tasks at the same time**, but **not necessarily running at the exact same moment**.
- Like a chef **switching between cooking different dishes** quickly.

Parallelism

- **Running multiple tasks at the exact same time** (using multiple cores).
- Like **multiple chefs cooking dishes at the same time**.

Trick to Remember

- **Thread = Worker inside a program**
 - **Concurrency = One worker switching tasks quickly**
 - **Parallelism = Many workers doing tasks at the same time**
-

Environment Variables

1. Environment Variables

- **Definition:** Key-value pairs used to configure system behavior and software.

- **Example:** `PATH`, `JAVA_HOME`, `NODE_ENV`
- **Use:** Control app behavior without changing code.

2. System Variables

- **Scope:** Apply to all users on the computer.
- **Set By:** OS or system admin.
- **Where to set (Windows):** Control Panel > System > Environment Variables > System variables.
- **Example:** `PATH`, `TEMP`

3. User Variables

- **Scope:** Apply only to the current user.
- **Use:** Custom settings for a specific user.
- **Where to set (Windows):** Control Panel > System > Environment Variables > User variables.
- **Example:** `JAVA_HOME`, `NODE_PATH`

4. Process Variables

- **Scope:** Available only during a running process.
- **Set In:** Code or terminal before launching a process.
- **Example (Node.js):**

```
console.log(process.env.NODE_ENV);
```

- **Temporary:** Lost when the process ends.

Tip:

- In terminal (Linux/macOS):

```
export MY_VAR=value
```

- In Windows CMD::

```
set MY_VAR=value
```

`.env` File (for development)

- Stores environment variables safely in key-value format.
- Example:

```
DB_HOST=localhost  
DB_PORT=5432
```



Installing WSL (Windows Subsystem for Linux)



What is WSL?

WSL lets you run a Linux terminal and tools on Windows without a virtual machine.



Steps to Install WSL (Recommended: WSL 2)



Step 1: Open PowerShell as Administrator

- Right-click Start → select **Windows PowerShell (Admin)**



Step 2: Install WSL

```
wsl --install
```

This command:

- Enables required features
- Downloads the Linux kernel
- Sets WSL 2 as default
- Installs Ubuntu by default



Step 3: Restart Your PC

- You'll be prompted to restart after installation.



Step 4: Set Up Linux

- After restart, a Linux terminal (e.g., Ubuntu) opens
- Set your **username** and **password**



Optional Commands

- 💡 **Check WSL version:**

```
wsl --list --verbose
```

- 📄 **Set default WSL version to 2:**

```
wsl --set-default-version 2
```

- 📦 Install a specific distro:

```
wsl --install -d Debian
```

- 🔄 Update WSL kernel:
Download from: <https://aka.ms/wsl2kernel>

🔧 Requirements

- Windows 10 (version 2004 and later) or Windows 11
- Virtualization enabled in BIOS

📁 Folder, 📄 File & 🔗 Symbolic Link

📁 Folder (Directory)

- Container for files and subfolders.
- Organizes data hierarchically.
- Examples:
 - Windows: `C:\Users\Afnur\Documents`
 - Linux: `/home/afnur/Documents`

📄 File

- Holds data (text, code, images, etc.)
- Named with an extension like `.txt`, `.jpg`, `.py`.
- Examples: `notes.txt`, `report.pdf`

🔗 Symbolic Link (Symlink)

- A reference or shortcut to another file/folder.
- Does not store data.
- Linux command: `ln -s target link`
- Windows command: `mklink link target`
- Example: `/home/afnur/shortcut.txt` → `/home/afnur/data.txt`

🔍 Differences Summary

Feature	Folder	File	Symbolic Link
Contains	Files/Folders	Data	Pointer
Directly usable	✓	✓	✓ (if target exists)

Feature	Folder	File	Symbolic Link
Real data	✗	✓	✗
Editable	✓	✓	✓ (affects target)

File System & Path System: Windows vs Linux

File System:

Feature	Windows	Linux
File System Type	NTFS, FAT32	ext4, Btrfs, XFS
Case Sensitivity	No (<code>file.txt</code> = <code>File.txt</code>)	Yes (<code>file.txt</code> ≠ <code>File.txt</code>)
Drive Structure	Multiple drives (C:, D:)	Single root /
Root Directory	C:\	/
Permissions	Basic (R/W/X)	Advanced (User, Group, Others)
Hidden Files	Hidden attribute	Start with <code>.</code> (e.g. <code>.bashrc</code>)

Path System:

Feature	Windows	Linux
Path Separator	\ (backslash)	/ (forward slash)
Example Path	C:\Users\Afnur\Documents	/home/afnur/Documents
Env Variables	%PATH%, %USERPROFILE%	\$PATH, \$HOME
Relative Path	.\file.txt	./file.txt
Absolute Path	C:\path\to\file	/path/to/file

Key Points:

- Linux is case-sensitive and developer-focused.
- Windows uses drive letters and is more user-focused.
- Use correct path format in cross-platform scripting.

Executable Files

What is an Executable File?

An **executable file** is a file that tells the computer to **perform a task or run a program** when opened.

Common Executable File Types:

OS	Extension	Example
Windows	<code>.exe</code> , <code>.bat</code>	<code>setup.exe</code>
Linux/Unix	No extension or <code>.sh</code>	<code>install.sh</code>
MacOS	<code>.app</code>	<code>Safari.app</code>

How It Works:

- Contains **binary code** (machine-readable instructions).
- Launched by **clicking** or using the **command line**.
- Created using **compilers** (like C, C++, etc.).

Safety Tips:

- ⚠ Don't run executables from unknown sources.
- Use antivirus to scan `.exe` files.
- Be careful with `.bat` or `.sh` scripts – they can run harmful commands.

How to Make One?

C/C++ example:

```
#include <stdio.h>
int main() {
    printf("Hello, world!");
    return 0;
}
```

Compile it using:

```
gcc hello.c -o hello.exe    (Windows)
gcc hello.c -o hello        (Linux/Mac)
```

Where Are They Used?

- Software installers
- Games
- System tools
- Scripts for automation

Extra:

On Linux/Mac: Make a file executable using:

```
chmod +x filename.sh
```

☑ Summary:

Executable files are programs ready to run. They're powerful and useful, but also risky if not handled carefully.

File Permissions in Linux 🛡️

📁 What are File Permissions?

File permissions control **who can read, write, or execute** a file or directory in Linux/Unix systems.

🔑 Three Types of Permissions:

- **r** → Read (view file contents)
- **w** → Write (modify file)
- **x** → Execute (run as a program/script)

👤 Three Types of Users:

1. **User (u)** – The file owner
2. **Group (g)** – Users in the file's group
3. **Others (o)** – Everyone else

📄 Viewing Permissions:

Use `ls -l`

Example:

```
-rwxr-xr-- 1 user group 1234 May 23 myfile
```

```
[first character = '-' mean files, 'd' mean directory, 'l' mean symbolic link]
```

Breakdown:

- `rw` → user
- `-x` → group
- `--` → others

📁 Symbolic Permission System:

Format: `chmod [user]+/-[permission] file` or `chmod +/-[permission] file` (for all u, g, o)
except `+w`

Examples:


```
chmod u+x script.sh    # Add execute to user
chmod g-w file.txt      # Remove write from group
chmod +x script.sh      # Add execute to user, group, other
chmod +w file.txt       # Add write to user (!group & other)
```

Numeric Permission System:

Each permission has a value:

- $r = 4, w = 2, x = 1$

Combine them:

- $7 = rwx, 6 = rw-, 5 = r-x, 4 = r--$

Format: `chmod [number] file`

Example:

```
chmod 755 myfile
# Means: rwxr-xr-x
```

Directory Permissions:

- **r** – List files
- **w** – Add/delete files
- **x** – Enter directory (`cd` into it)

Tips:

- `chmod 777` = full access to everyone (⚠ risky!)
- Use `chmod`, `chown`, `chgrp` to manage permissions, ownership, and groups.

Command Execution Order in Bash

When you type a command in Bash, it checks for matches in this order:

1. Alias

Checks if the command is an alias (a shortcut for another command).

Example: `ll` → `ls -l`

2. Functions

Checks for a shell function with the same name.

Functions are like mini scripts defined in the shell.

3. 🔑 Built-in Commands

Checks if it's a built-in shell command (like `cd`, `echo`, `pwd`).

4. ⚡ Hash Table

Bash uses a cache (hash table) to quickly find previously used executables.

5. 📁 Executable Files in PATH

Finally, searches the directories in the `$PATH` environment variable for a matching program.

📄 Example

If you type `ls`, Bash will:

1. See if `ls` is an alias → if yes, use it.
2. If not, check for a function named `ls`.
3. Then check if `ls` is a built-in.
4. Then check its hash table.
5. Finally, search for `ls` in folders like `/bin`, `/usr/bin`.

🔑 Important Methods and Properties of `process` Object in Node.js

🔑 Properties

1. `process.argv`
 - Array of command-line arguments passed when running the script.
2. `process.env`
 - Object containing environment variables.
3. `process.pid`
 - Process ID of the current Node.js process.
4. `process.ppid`
 - Parent process ID.
5. `process.platform`
 - OS platform (e.g., `win32`, `linux`).
6. `process.version`
 - Node.js version.
7. `process.versions`

- Versions of Node.js and its dependencies.

8. `process.arch`

- Processor architecture (e.g., `x64`).

Methods

1. `process.cwd()`

- Returns the current working directory.

2. `process.chdir(path)`

- Changes the current working directory.

3. `process.memoryUsage()`

- Returns memory usage info.

4. `process.uptime()`

- Returns how long the process has been running (in seconds).

5. `process.exit([code])`

- Ends the process with the given exit code.

6. `process.kill(pid)`

- Sends a signal to terminate a process (usually itself).

7. `process.emitWarning(msg, options)`

- Emits a custom warning.

8. `process.nextTick(callback)`

- Schedules a function to run on the next event loop tick.

Stream Interaction

- `process.stdin` – Input stream (like keyboard).
- `process.stdout` – Output stream (standard output).
- `process.stderr` – Error stream.

Event Listeners

- `process.on("exit", callback)` – Triggered before the process exits.
- `process.on("warning", callback)` – Triggered when a warning is emitted.